The Angular edition

How angular.mvcprogrammer.com is built, how it differs from the vanilla edition at mvcprogrammer.com, the Blazor edition at blazor.mvcprogrammer.com, and the React edition at react.mvcprogrammer.com, and the rules for changing, testing, and deploying it.

1. Why there are four editions

The portfolio ships four times. The default edition is plain HTML, CSS, and JavaScript at the repository root, served at mvcprogrammer.com. The Blazor edition is a Blazor WebAssembly app under `blazor/`, served at blazor.mvcprogrammer.com and documented in docs/BLAZOR-EDITION.pdf. The Angular edition is an Angular 22 workspace under `angular/`, served at angular.mvcprogrammer.com. The React edition is a React 19 app built with Vite under `react/`, served at react.mvcprogrammer.com and documented in docs/REACT-EDITION.pdf. All four show the same content, load the same stylesheet under their own palettes, and call the same Experience API. The Angular edition exists to show that content consumed from standalone components, signals, and a reactive form, next to the React, C#, and vanilla versions.

The one rule that matters

Every content, markup, behavior, or styling change made to one edition is made to the other three in the same commit. Before finishing any change to the site, check the other editions for the same change. Section 4 lists which files are shared (one edit) and which are duplicated (four edits).

No exceptions

Content is identical on every edition. The editions cross-link through the About core-skill chips (**Angular**, **React**, **Blazor**, **JavaScript**, each with a title hint such as See the Angular edition of this site) and the hero highlights (**Angular** and **React** of Angular / React to their editions), and through the edition menu in the header: a pill naming the open edition that lists all four. A link to the edition already open simply reloads it. The portfolio card carries no edition links. The spec in `sections/static-sections.spec.ts` pins the Angular side of it.

The one exception: color

Each edition carries its own palette. One small file per edition, loaded after the shared `styles.css`, overrides only the design tokens (`--bg`, `--accent`, `--hover`, and the rest) with `light-dark()` values, so one declaration covers both color schemes and no rule from the shared stylesheet is repeated. Older browsers ignore `light-dark()` and fall back to the shared JetBrains Rider palette. The four palettes, with the page, the accent, and the hover color for each scheme:

Edition	Palette file	Palette	Light	Dark
Default	`vanilla.css`	Eggshell paper with a terracotta accent and a coral hover; a warm charcoal after dark.		
Angular	`angular/src/angular.css`	Navy and brass: a cool, faintly blue page with navy buttons and a goldenrod hover; a true navy after dark with muted brass for buttons and links and a paler gold on hover.		
React	`react/src/react.css`	The react.dev documentation palette: cool slate surfaces, React blue for buttons and links, the logo cyan on hover.		
Blazor	`blazor/src/Portfolio.Blazor/wwwroot/blazor.css`	.NET violet: the .NET brand purple on faint lavender, a deep violet-black with lilac buttons after dark, the Blazor gradient's pink on hover.		

The Angular build emits `angular.css` under the bundle name `angular` (the `bundleName` option in `angular.json`) so that `ng serve` does not name it `styles.css` and shadow the shared file; in production the name is hashed either way.

Every edition's colophon describes its own palette, and the four are listed with screenshots in the README of this repository and on the GitHub profile README.

2. Where things live

The workspace follows the folder-by-kind layout from the coding standards, in Angular's dress: plural folder names under `src/app/`, the Angular 20+ file naming (no `.component` or `.service` suffixes), and one spec file beside each source file.

Path	What it holds
`angular/`	The workspace: one application project, `portfolio-angular`, zoneless, strict TypeScript, Vitest.
`scripts/sync-shared.mjs`	Copies the shared root files into `public/` before every build and serve (the `prebuild` and `prestart` npm hooks). `public/` is generated and not committed.
`src/index.html`	The shell: metadata, the shared stylesheet link, the boot screen inside `<app-root>`. The build appends its own script and stylesheet tags.
`src/angular.css`	The only extra stylesheet: this edition's palette (section 1), the boot screen, `display: contents` for every component host element, and form validation state. Emitted under the bundle name `angular`.
`src/app/app.ts`	Layout: skip link, icon sprite, header, `<main>` with the router outlet, footer.

Path	What it holds
`src/app/app.config.ts`	Router, global error listeners, and `provideSite()` with the endpoints.
`src/app/app.routes.ts`	`""` is Home; `**` is NotFound, for the paths CloudFront hands to `index.html`.
`pages/`	`Home` (the sections in page order) and `NotFound`.
`sections/`	One component per section of the page: Hero, About, Quote, Services, Freelance, Portfolio, Resume, Contact, Colophon. Templates lifted from `index.html`.
`components/`	Header (menu, scroll state, current section), footer, icon and icon sprite, social links, the experience timeline with its skill filter, and the kiosk demo dialog.
`configurations/`	`SiteOptions` and its `SITE_OPTIONS` injection token.
`interfaces/`	`ExperienceClient`, `ContactClient`, `ContactTopicState`: abstract classes that double as injection tokens, so components depend on the contract and tests swap in fakes.
`services/`	`HttpExperienceClient` and `HttpContactClient` over `HttpClient`, and `SubjectContactTopicState`.
`providers/`	`provideSite()`: validates the options, then registers HttpClient over fetch, the options, the two clients, and the topic state. The counterpart of the Blazor edition's `AddPortfolioServices()`.
`models/, requests/`	Interfaces matching `api/openapi.yaml` and the contact Lambda's JSON body.
`enumerations/`	`WorkMode`, a string enum matching the API's kebab-case values.
`pipes/`	`monthLabel` (2023-12 to Dec 2023) and `placeLabel` (Remote, Hybrid, or the place).
`validators/`	`phoneRequiredForCall`, the contact form's one cross-field rule, with the `REPLY_BY_PHONE` constant.
`src/testing/`	Fakes of the two clients, fixture factories, the test options, and DOM helpers shared by the specs. Section 6.
`angular/README.md`	The short version of this document, kept in the repo.

The workspace is not in `mvcprogrammer.com.sln`; open `angular/` as its own project in the IDE.

3. How the Angular edition differs from the others

The visible page is the same. The differences are in how it gets there. The React edition is compared column by column in its own guideline.

Concern	Vanilla edition (root)	Blazor edition (blazor/)	Angular edition (angular/)
Rendering	Hand-written HTML; reads fully without JavaScript.	Razor components on .NET in WebAssembly; a boot screen until the runtime loads.	Standalone components, OnPush, zoneless; a short boot screen until the 95 kB (compressed) bundle runs.
Where state lives	DOM attributes and closures in `main.js`.	C# fields in the code-behinds.	Signals on the components; the templates read them, `computed()` derives labels and paths.

Concern	Vanilla edition (root)	Blazor edition (blazor/)	Angular edition (angular/)
JavaScript	`main.js`, about 470 lines.	`site.js`, about 100 lines of browser APIs behind JS interop.	None outside the components. They call `IntersectionObserver`, `matchMedia`, and `<dialog>` directly, cleaning up through `DestroyRef` and effect cleanups.
Experience timeline	In the HTML; re-rendered from the API when it answers, HTML stays on failure.	`ExperienceTimeline` loads from the API, falls back to the bundled seed.	Same component and same fallback, written with `async/await` over Promise-returning clients and wrapped in `PendingTasks` so tests and the router can wait on it.
Skill chips	Pinned, rest, demoted; twelve before the more button.	Same constants in C#.	Same constants as statics on `ExperienceTimeline`. Pinning a different skill means editing all four.
Contact form	HTML5 validation plus `checkValidity()`; fetch.	`EditForm` with data annotations; typed HttpClient.	A typed `FormBuilder` group with the same rules; the phone control is enabled only for a call, so a hidden field never validates. Same status messages, including the timeout one.
Topic preselection	Buttons set the select's value.	`IContactTopicState` singleton.	`ContactTopicState`, a Subject the buttons push to and the form subscribes to with `takeUntilDestroyed()`.
Configuration	`data-` attributes in the HTML.	`appsettings.json` bound to `SiteOptions`.	A `SiteOptions` literal in `app.config.ts`, validated by `provideSite()` and injected through `SITE_OPTIONS`.
Header behavior	Listeners in `main.js`.	Observers in `site.js` reporting through `[JSInvokable]`.	Observers in `SiteHeader`; the section observer attaches after each `NavigationEnd` because the sections render after the header.
Kiosk demo	Native dialog; iframe cleared on close.	`KioskDialog` with JS interop for `showModal`.	`KioskDialog` calls `showModal()` itself; the iframe exists only while the `open` signal is true.
Styling	Shared stylesheet plus `vanilla.css` (palette).	Shared stylesheet plus `blazor.css` (palette, boot screen).	Shared stylesheet plus `angular.css` (palette, boot screen). **No component styles**: Angular injects them inline, which the CSP forbids. Host elements get `display: contents` so the sticky header and the grids are unaffected.
Build output	The files as committed.	`dotnet publish`, trimmed.	`ng build`: esbuild, fingerprinted bundles, critical-CSS inlining off (it would be an inline `<style>`).
Hosting	Bucket root, distribution for mvcprogrammer.com.	`blazor/` prefix, own distribution, CSP with `'wasm-unsafe-eval'`.	`angular/` prefix, own distribution, the static site's response headers policy unchanged, S3 403 rewritten to `index.html`.

Concern	Vanilla edition (root)	Blazor edition (blazor/)	Angular edition (angular/)
Tests	None for the static page.	118 bUnit and xUnit cases.	138 Vitest cases on jsdom, run before every build.

4. Shared by copying versus duplicated by hand

Angular refuses asset paths outside the workspace, so the shared files cannot be linked the way the Blazor project file links them. `scripts/sync-shared.mjs` copies them into `public/` before every build and serve instead, and `public/` stays out of git, so there is still one source on disk. Everything else is written once per edition.

Edit once (copied into public/)

- `styles.css`, the whole stylesheet, linked from `index.html` by hand, ahead of the build's own stylesheet. The palette on top of it is per edition (section 1), so a color change is four edits.
- `assets/**`: fonts (Inter and JetBrains Mono as variable WOFF2 subsets) and images.
- `favicon.svg`, `favicon.ico`, `apple-touch-icon.png`.
- `api/seed/positions.json`, served to the Angular edition as `data/positions.json`.

Edit four times (one copy per edition)

- Markup: `index.html`, the Razor sections and components, the templates under `angular/src/app/sections/` and `angular/src/app/components/`, and the React edition's JSX under the same folder names.
- Behavior: `main.js`, the Razor code-behinds plus `site.js`, the Angular component classes, and the React components and hooks.
- Constants that shape the page: pinned and demoted skills, the visible chip count, status messages, section ids and labels.
- Endpoints: `data-` attributes in `index.html`, `wwwroot/appsettings.json`, the `siteOptions` literal in `app.config.ts`, and the one in the React edition's `main.tsx`.

When a change touches a duplicated file, the commit is not finished until all four copies match. When the Angular copy changes behavior, add or update the spec that proves it.

5. Running it locally

Node 24 and npm; the .NET SDK is not needed. All commands run from `angular/`.

```
npm ci
npm start                    # http://localhost:4200, copies the shared files first
npm run build                # dist/portfolio-angular/browser/
```

The dev server has no `/api`, so the timeline falls back to the bundled seed and the skill filter stays hidden. To see the filter, serve the build output behind the real CloudFront behaviors or any host that proxies `/api` to the Experience API. The dev server also injects its own inline scripts for live reload, which is fine locally; the production build has none.

The contact Lambda rejects Origins it does not know, so the form cannot send from a local preview in any edition.

6. Testing

The suite is Vitest 4 through the Angular CLI's unit-test builder, on jsdom, with the Vitest globals. Components render through `TestBed` against fakes of the two client interfaces, so the skill filter, the seed fallback, the contact form's validation and status messages, the mobile menu, the section observer, and the kiosk dialog are all exercised without a browser. The two HTTP clients are tested over `HttpTestingController`. The deploy workflow runs the suite before every build, so a red test blocks the Angular deploy.

Commands

```
# run the suite once
npm test
# re-run on change
npm run test:watch
# one suite or one test, by name (a regular expression)
npx ng test --watch=false --filter "ContactSection"
npx ng test --watch=false --filter "should fall back to the seed"
# one file
npx ng test --watch=false --include "**/site-header.spec.ts"
```

Conventions the suite follows

- One `*.spec.ts` beside each source file. `describe` names the class, a nested `describe` names the method, and `it` states the behavior in a sentence, so the report reads like the C# names `Method_ShouldExpectedResult`.
- Arrange, Act, Assert, each marked with a comment. Scenario setup across the fakes lives in `arrange...` helpers at the bottom of the file; fixtures come from `create...` factories in `src/testing/test-data.ts`.
- Dependencies of the class under test are faked, never the class itself. `createSiteTestServices()` builds the fakes, the real topic state, and the test options in one call.
- Zoneless rules: every async step is followed by `await fixture.whenStable()`. The components wrap their loads in `PendingTasks`, which is what makes that wait reliable.
- jsdom has no `IntersectionObserver`, no `matchMedia`, and no modal dialogs. The components guard for that; the specs stub the observer and the dialog methods where the behavior matters.

Test layout

Files mirror the app. The counts are test cases as Vitest reports them (138 in all).

File	Tests	What it proves
components/experience-timeline/experience-timeline.spec.ts	16	API render, date and place labels, every fallback path (error, empty, not a list, seed failure, empty seed, skills failure), chip order, the more button, chip titles, filtering, restore without refetch, filter failure, late response after destroy.
components/site-header/site-header.spec.ts	19	Nav links and initial state, the edition list (four entries, current marked, open and close, Escape, outside click), menu toggle, close on nav click, Escape with focus return, outside and inside clicks, listeners released, scrolled class, section observer after navigation, current link set and cleared, observer replaced and disconnected, no observer in the browser.

File	Tests	What it proves
components/kiosk-dialog/kiosk-dialog.spec.ts	7	No iframe while closed, show renders the configured kiosk and opens modally, show is idempotent, close button, close before open, backdrop versus inside click, close event removes the iframe.
components/icon-sprite/icon-sprite.spec.ts	14	Every symbol the page references exists in the sprite.
components/icon, social-links, site-footer	3	Icon references a sprite symbol, three social profiles, footer year and legal links.
sections/contact-section/contact-section.spec.ts	15	Phone field hidden and disabled by default, topic preselection and unsubscribe, phone reveal and hide, post and confirmations, honeypot forwarded, 429, generic failure, timeout, non-HTTP error, no post when invalid, when a call has no number, or the message is too short.
sections/static-sections.spec.ts	6	Hero, the About tag exception, the quote, the portfolio links and kiosk button, the resume URL and education, the colophon.
sections/call-to-action.spec.ts	2	The Services and Freelance buttons select their contact topic.
pages/home, pages/not-found, app.spec.ts	5	Home renders every section in order, NotFound, the layout around the routed page, an unknown path routes to NotFound.
services/beacon-engagement-tracker.spec.ts	10	View pings once per section, leave pings with the visible seconds, silence on localhost and under Do Not Track or Global Privacy Control, no observer in the browser, failed pings ignored, restart and stop.
services/http-experience-client.spec.ts	9	Paths, timeout option, query-string escaping, blank skill ignored, error status, null bodies, skills, seed.
services/http-contact-client.spec.ts	3	JSON post to the endpoint with the Accept header and timeout, empty success body, status code on rejection.
services/subject-contact-topic-state.spec.ts	4	Notify every subscriber, no subscribers, unsubscribe stops notifications, no replay.
providers/site-providers.spec.ts	6	Everything registered, one topic state, invalid options rejected before providing, validation messages.
pipes/, validators/	19	Month labels and unparseable input, place label per work mode, the phone rule in every reply state.
src/testing/		`FakeExperienceClient` and `FakeContactClient` (scriptable, record their calls), `createSiteTestServices()`, `TEST_SITE_OPTIONS`, fixture factories, and the `type`, `choose`, and `textOf` DOM helpers.

7. Deploying

The `deploy-angular` job in .github/workflows/deploy.yml runs on every push to `main`, in parallel with the other three jobs. It sets up Node 24 with the npm cache, runs `npm ci`, `npm test`, and `npm run build` (which copies the shared files first), assumes the deploy role through OIDC, and syncs the output in two passes before invalidating the distribution.

- The fingerprinted bundles (`main-*.js`, `polyfills-*.js`, `chunk-*.js`, `angular-*.css`) get a one-week browser TTL and `immutable`.
- Everything else, `index.html`, the shared stylesheet, icons, images, and the seed, gets a five-minute browser TTL.
- CloudFront compresses on the fly; nothing is pre-compressed.

A manual deploy is the same two syncs plus an invalidation; the exact commands, the resource creation steps, and the distribution, certificate, Route 53, bucket policy, and IAM details are in the Angular section of docs/DEPLOY.md. Resource IDs are documented there once and are not repeated here.

8. Checklist for a change

Use this before marking any site change done.

Step	Do
1	Make the change in the edition you started with.
2	If the file is shared (section 4), you are done with content. If it is duplicated, make the matching change in the other three editions.
3	If the Angular behavior changed, add or update the spec that proves it. Follow the conventions in section 6.
4	Run `npm test` and `npm run build` in `angular/` and make both green. Warnings in the build are errors in review.
5	Preview the editions locally: `python3 -m http.server 8080` at the root, `dotnet run --project blazor/src/Portfolio.Blazor`, and `npm start` in `angular/` and in `react/`.
6	Check the cross-links survived: the edition menu in the header, the four About chips, and the two hero highlights, identical on every edition, with their hints.
7	If you added a component, add its selector to the `display: contents` list in `angular.css`. Never add component styles. A color change edits the palette block at the top of the same file, and the other three palette files if it is meant to be shared.
8	Commit all editions together. No AI attribution trailers in the message.
9	Push to `main` only when asked. The workflow tests, builds, and deploys every edition.

9. Links

What	Where
Live, Angular edition	https://angular.mvcprogrammer.com
Live, Blazor edition	https://blazor.mvcprogrammer.com
Live, vanilla edition	https://mvcprogrammer.com
Repository	https://github.com/mvcprogrammer/mvcprogrammer.com
Angular workspace	angular/
Angular app source	angular/src/app
Angular README	angular/README.md
Blazor guideline	docs/BLAZOR-EDITION.pdf
Live, React edition	https://react.mvcprogrammer.com
React guideline	docs/REACT-EDITION.pdf
Deploy walkthrough	docs/DEPLOY.md
Deploy workflow	.github/workflows/deploy.yml
Experience API spec	api/openapi.yaml
Contact Lambda	infra/lambda/contact/index.mjs
Workflow runs	GitHub Actions